Content-Based Filtering

A Study Guide to the Deep-Learning Recommender-Systems Lectures

Andrew Ng – Machine Learning Specialization, Course 3 · Unsupervised Learning, Recommenders & Reinforcement Learning

Five lectures, reconstructed section by section: the pivot from collaborative filtering, the two-tower deep-learning architecture, scaling with retrieval & ranking, the ethics of recommender systems, and the TensorFlow implementation.

1. Collaborative → Content-Based
2. Deep Learning Architecture
3. Retrieval & Ranking at Scale
4. Ethics of Recommender Systems
5. TensorFlow Implementation
6. Interview Prep & Real-World Use
7. Tailored Report

1. Why a New Algorithm? From Collaborative to Content-Based Filtering

Andrew never introduces content-based filtering in a vacuum — he opens by contrasting it against collaborative filtering, which you already know, and only then shows exactly where the two diverge. This anchoring move is his standard pattern.

The contrast he draws

- **Collaborative filtering** recommends items based on *ratings of users who rated similarly to you*. It needs only the ratings matrix — no descriptive features.
- **Content-based filtering** recommends items based on *features of the user* and *features of the item*, matched against each other. It needs feature vectors x_u and x_m in addition to the ratings.
- Notation carried over unchanged: r(i,j) = did user j rate item i, y(i,j) = the rating given — a deliberate continuity signal, not new material.

Building the feature vectors — his examples

Whose features	Examples Andrew gives
User — x_u	age · gender (one-hot) · country (one-hot, ~200 values) · watched-top-1000-movies indicator · average rating given per genre
Movie — x_m	year of release · genre(s) · critic review score(s) · average rating the movie received (overall / by country / by demographic)

NOTE — Worth remembering: *features like "average rating per genre" or "average rating of this movie" are themselves derived from the ratings data. Andrew explicitly says there is nothing wrong with a feature vector depending on labels used elsewhere in the system — a subtlety that can look like leakage if you're not used to recommender-system feature engineering.*

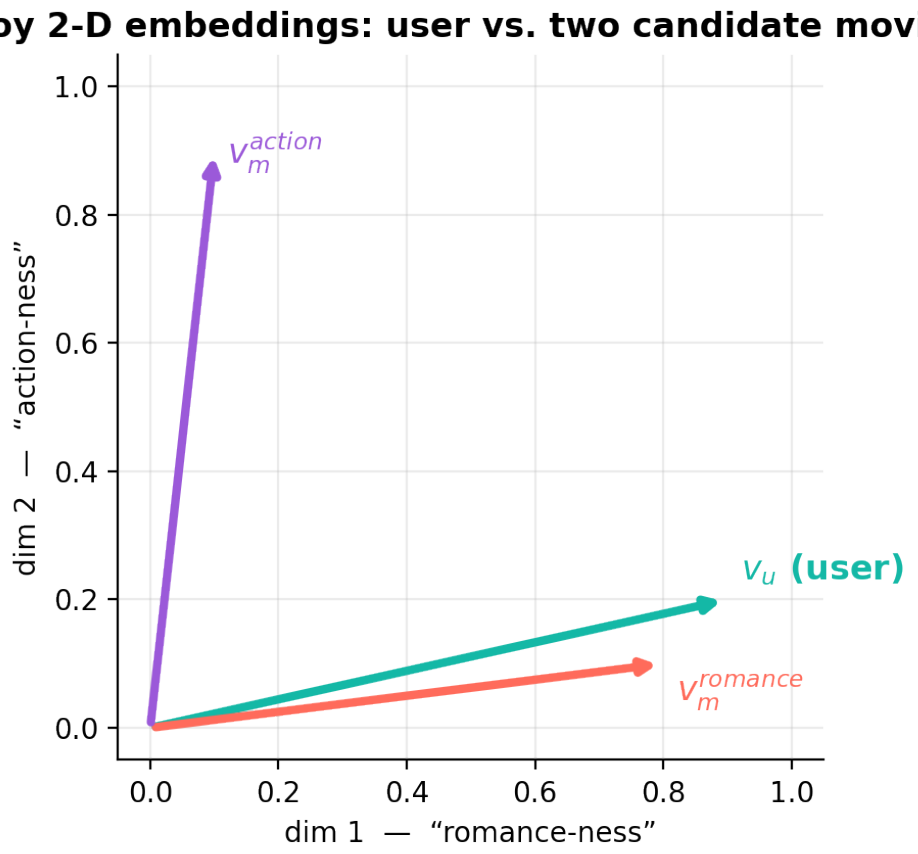
From w·x + b to $v_u \cdot v_m$

He rewrites the old collaborative-filtering prediction $\mathbf{w_j \cdot x_i + b_j}$ in two moves: (1) drop b_j entirely — he asserts this doesn't hurt performance — and (2) rename w_j and x_i as **learned** vectors v_u^j and v_m^i.

Critical constraint: x_u and x_m can be different lengths (his example: 1,500 numbers for a user vs. 50 for a movie), but $\mathbf{v_u}$ **and** $\mathbf{v_m}$ **must be the same length** (his running example: both length 32) — a dot product requires matching dimensions.

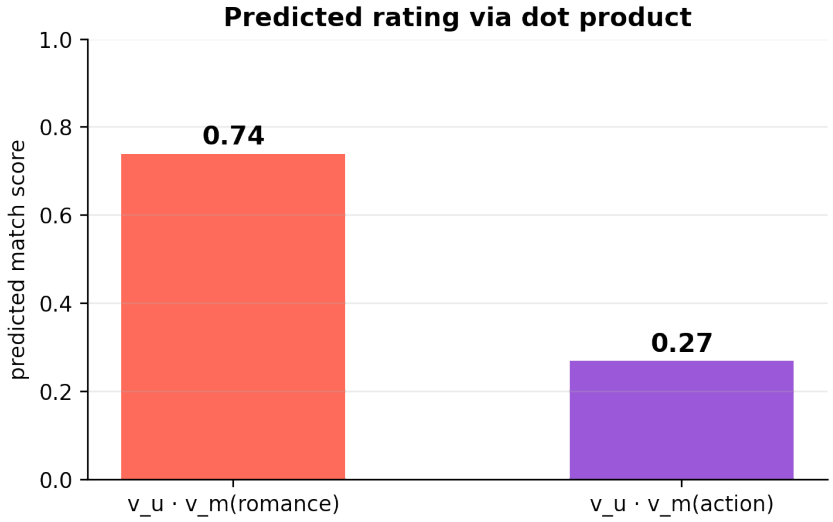
Simple worked example (toy 2-D case)

Real embeddings are 32 numbers, but the mechanics are easiest to see in 2-D. Suppose dimension 1 loosely tracks "romance-ness" and dimension 2 tracks "action-ness" (Andrew's own illustrative framing — the real learned numbers aren't literally labeled this way, but it builds intuition).



Vector	Values	Meaning
v_u (user)	[0.90, 0.20]	strongly prefers romance, mildly likes action
v_m (romance movie)	[0.80, 0.10]	mostly romance content
v_m (action movie)	[0.10, 0.90]	mostly action content

Predicted rating = $v_u \cdot v_m$ (element-wise multiply, then sum):
romance movie: (0.90×0.80) + (0.20×0.10) = 0.72 + 0.02 = **0.74**
action movie: (0.90×0.10) + (0.20×0.90) = 0.09 + 0.18 = **0.27**





The romance movie scores far higher — the dot product rewards alignment between a user's taste-vector and a movie's content-vector, exactly the intuition Andrew builds before handing off to the deep-learning lecture.

Open question this sets up

Given x_u, how do you *compute* v_u? Given x_m, how do you compute v_m? This cliffhanger is intentional — it's exactly what the deep-learning lecture answers next.

Takeaway: Content-based filtering replaces the linear model $w_j \cdot x_i + b_j$ with a dot product of two *learned*, equal-length vectors v_u and v_m, computed from (possibly very different-length) feature vectors x_u and x_m.

2. Computing v_u and v_m with Deep Learning

Andrew frames this as *the* way modern commercial content-based filtering systems are actually built today — not a toy alternative.

Two parallel networks (the "two-tower" model)

- **User network**: input x_u → several dense layers → output layer of **32 units** (unusual relative to the 1-unit output networks used earlier in the course) → this output *is* v_u.
- **Movie / item network**: input x_m → several dense layers → output layer, also 32 units → this is v_m.
- The two networks can have *different* numbers of hidden layers and hidden units per layer — but the **output width must match**, since the dot product needs equal-length vectors.

Cost function — compared directly to collaborative filtering's

NOTATION (PSEUDOCODE)

```
# same shape as collaborative filtering's cost, but v_u/v_m are now
# the OUTPUTS of two neural networks, not directly-learned parameters
J = sum(
(dot(v_u[j], v_m[i]) - y[i][j]) ** 2
for i, j in rated_pairs # where r(i, j) == 1
)
# + regularization term on all network weights
```

He is explicit: there is **no separate training procedure** for the user network and the movie network. A single cost function trains *all* parameters of *both* networks jointly via gradient descent (or another optimizer) — easy to assume otherwise, but wrong.

Binary-label variant

If y is binary (like/favorite rather than a star rating), apply a **sigmoid** to $v_u \cdot v_m$ to predict $P(y^{ij}=1)$ — directly parallel to how logistic regression wraps a sigmoid around a linear score.

Finding similar items (a bonus use of the trained model)

Once trained, v_m^i is a fixed 32-length descriptor for movie i. To find movies similar to movie i, search for movies k with small squared distance $||v_m^k - v_m^i||^2$ — the same trick used with collaborative-filtering features, just on the learned v_m instead.

NOTE — Forward-looking aside (seeds Section 3): *because $v_m^{(i)}$ doesn't depend on any particular user,* ***all movie vectors can be pre-computed offline*** *— e.g. overnight on a batch server — and cached in a lookup table. Andrew says this pre-computability "will turn out to be important later when we talk about scaling."*

Where the real engineering effort goes

A practical aside: in industry, most developer time is spent not on network architecture but on **feature engineering** for x_u and x_m — a commercial-reality caveat, not something formally testable.

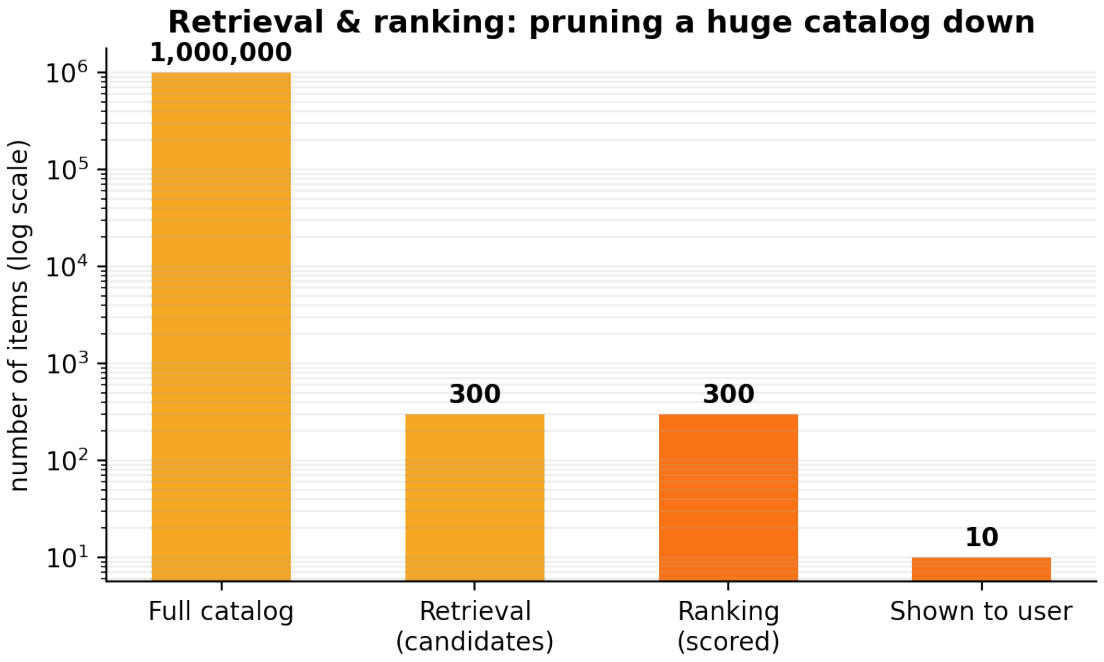
Named limitation — sets up the next lecture

Running the full network for every item in a catalog of millions is **computationally infeasible**. This limitation is the explicit motivation for the retrieval-and-ranking architecture that follows.

Takeaway: The two-tower architecture learns v_u and v_m jointly through one shared MSE cost function; because v_m doesn't depend on the user, it can be pre-computed offline for every item — the single fact that makes both "find similar items" and large-scale retrieval possible.

3. Scaling Up: The Retrieval & Ranking Architecture

Motivating scale numbers Andrew gives: thousands of movies, millions of ads, tens of millions of songs, millions-to-tens-of-millions of products. Running a full forward pass against every catalog item on every page load is computationally infeasible — so large systems split the job into two steps.



Retrieval step — goal: broad coverage, cheaply

It's fine if the retrieval step includes items the user won't like — the only sin here is *missing* good candidates. Andrew's concrete recipe for movies:

1. For each of the user's last 10 watched movies, look up the 10 most similar movies (using the **pre-computed** v_m nearest-neighbor tables from Section 2).
2. Add the top 10 movies from each of the user's top 3 most-watched genres.
3. Add the top 20 movies in the user's country.
4. Merge everything into one list, **remove duplicates**, and **remove items already watched/purchased**.

Result: a shortlist of roughly "100s" of plausible candidates instead of the whole catalog.

Ranking step — goal: precision on the shortlist

Take the retrieval shortlist and run it through the **trained** two-tower network to compute an actual predicted rating/probability for each user–item pair, then sort and display by predicted score.

NOTE — Efficiency trick: *because v_m for every candidate item is already pre-computed, the only forward pass needed live is the user network, run **once**, to get v_u for the user who just showed up. Everything else is just dot products against cached vectors.*

Design knob: how many items to retrieve?

More retrieved candidates → better final recommendations but slower system. Andrew recommends **offline experiments**: check whether retrieving 500 items instead of 100 meaningfully raises the average predicted P(y=1) or rating of the retrieved set. If it does, that argues for retrieving more, accepting some latency cost. This is an empirical trade-off to tune, not a closed-form answer.

Retrival → Recall

Takeaway: Retrieval optimizes for *recall* over a huge catalog cheaply; ranking optimizes for *precision* on a small shortlist using the expensive, accurate model. This cascade is how large systems get both speed and accuracy at once.

ranking → Precision

4. Ethical Issues in Recommender Systems

Andrew gives this its own standalone lecture rather than a footnote — signalling he considers it as important as the math. His method differs from the technical lectures: he lists possible optimization *goals*, then works through which are safe and which are dangerous using paired case studies.

Goals he lists, with his running commentary

Possible objective	His read
Show items likely to be rated 5 stars	seems fine
Show products likely to be purchased	seems fine
Show ads most likely to be clicked	gets nuanced once combined with advertiser bidding
Show products that generate the largest profit	potentially deceptive to users
Maximize watch time / time-on-site	potentially harmful — engaging ≠ beneficial

He deliberately does not resolve the ambiguity between goals 1–2 ("quite innocuous") and 3–5 ("may be just fine" but "could also be problematic") — that ambiguity is the whole point of the lecture.

Case study 1: advertising as an amplifier (his central example)

- **Travel industry (virtuous cycle):** a genuinely good travel company serves customers well → more profitable → can outbid competitors for ads → shown to more people → serves even more people well.
- **Payday-loan industry (vicious cycle):** the company best at extracting money from vulnerable customers → most profitable → outbids competitors for ads → more traffic → exploits more people.
- The bidding mechanism is *identical* in both cases — the outcome depends entirely on the underlying business, which the algorithm cannot distinguish from profitability alone.

Proposed (imperfect) amelioration: refuse ads from exploitative businesses — but Andrew immediately flags that **defining "exploitative" is itself very difficult**.

Case study 2: engagement maximization and toxic content

Maximizing watch-time/engagement has been reported to amplify conspiracy theories and hate/toxic content, because such content is disproportionately engaging. Proposed amelioration — filter hate speech, fraud, scams, certain violent content — but again, **defining the filtering criteria precisely is "surprisingly tricky."**

Case study 3: transparency

Many users assume a site recommends what *they'd* like, when it may actually be optimizing profit. Andrew frames this as a trust issue and recommends companies be transparent about their recommendation criteria — a values-based recommendation, not a technical fix.

His closing framing

No formula for making recommender systems ethical. His actual advice is procedural: invite diverse perspectives, debate design choices openly, and only build systems you believe genuinely benefit users and society — a deliberate register shift from every other lecture in the module.

> **Takeaway:** The algorithm's math is neutral; harm or benefit emerges from *what objective it's told to optimize* and the business it's applied to. Mitigations all require difficult, non-algorithmic value judgments.

5. Practice-Lab Walkthrough: TensorFlow Implementation

Andrew's approach here is not to re-derive theory but to walk line-by-line through the practice-lab code, mapping each Keras call back to the math from Section 2.

User network and item (movie) network

PYTHON — the two towers

```
num_outputs = 32

user_NN = tf.keras.models.Sequential([
tf.keras.layers.Dense(256, activation='relu'),
tf.keras.layers.Dense(128, activation='relu'),
tf.keras.layers.Dense(num_outputs), # 32 units -> raw v_u
])

item_NN = tf.keras.models.Sequential([
tf.keras.layers.Dense(256, activation='relu'),
tf.keras.layers.Dense(128, activation='relu'),
tf.keras.layers.Dense(num_outputs), # 32 units -> raw v_m
])
```

Hidden-layer activation: **ReLU** by default ("our default choice," not a hard rule).

Feeding inputs in and normalizing

PYTHON — l2 normalization

```
input_user = tf.keras.layers.Input(shape=(num_user_features,))
vu = user_NN(input_user)
vu = tf.linalg.l2_normalize(vu, axis=1) # <-- the step easy to miss

input_item = tf.keras.layers.Input(shape=(num_item_features,))
vm = item_NN(input_item)
vm = tf.linalg.l2_normalize(vm, axis=1)
```

NOTE — Easy to miss: *after computing v_u, an extra **L2-normalization** rescales it to unit length; the same is applied to v_m. Andrew states plainly this step isn't strictly required by the Section-2 math, but empirically "makes the algorithm work a bit better." TensorFlow's l2_normalize function is literally where the name comes from — a practical addition, not something you'd derive from the cost function.*

Combining the towers and assembling the model

PYTHON — dot product + compile

```
output = tf.keras.layers.Dot(axes=1)([vu, vm]) # v_u . v_m

model = tf.keras.Model([input_user, input_item], output)

cost_fn = tf.keras.losses.MeanSquaredError()
model.compile(optimizer=tf.keras.optimizers.Adam(0.01), loss=cost_fn)
```

- **tf.keras.layers.Dot** is a special Keras layer whose only job is to take the dot product of two vectors — unlike the Dense layers used everywhere else in the course.
- **tf.keras.Model([user_input, item_input], output)** ties both input branches and the dot-product output into one trainable model.
- **Loss = mean squared error** — matching the cost function J from Section 2 exactly.

His closing note: these are "the key code snippets," not the entire lab — he expects you to read the rest of the notebook to see how everything connects end-to-end. This is also the last video of the recommender-systems block; reinforcement learning is next.

Takeaway: The Keras implementation is a direct, literal translation of the two-tower math: two Sequential stacks, an L2-normalize step that isn't in the original derivation but helps in practice, a Dot layer for $v_u \cdot v_m$, and a plain MSE loss.

6. Interview Prep & Real-World Application

Collaborative Filtering vs. Content-Based Filtering

1. **Q:** When would you choose content-based filtering over collaborative filtering, and what does it cost you to make that choice?
2. **Q:** Why can content-based filtering handle the “cold-start” problem for new items/users better than pure collaborative filtering?

Core takeaway: Collaborative filtering only needs the ratings matrix; content-based filtering trades that simplicity for needing well-engineered feature vectors, in exchange for recommending brand-new items/users with no rating history.

Real-world use case: News/article recommenders lean on content-based signals (topic, recency, keywords) because new articles have zero interaction history the moment they're published.

Two-Tower (Deep Learning) Architecture

1. **Q:** Why must the output layers of the user and item networks have the same dimensionality, even if the input feature vectors don't?
2. **Q:** Why is there a single joint cost function instead of training the two networks separately?

Core takeaway: This is the industry-standard “two-tower model” — it decouples user/item representation learning while forcing a shared embedding space via the dot product, and item embeddings can be precomputed and cached.

Real-world use case: The same architecture underlies large-scale embedding retrieval in production (e.g. video-platform recommendation embeddings) and modern semantic search (query tower + document tower, matched by similarity).

Retrieval & Ranking

1. **Q:** Why is it acceptable for the retrieval step to be “sloppy” while the ranking step must be precise?
2. **Q:** How would you decide how many candidates to pull in retrieval for a real production system?

Core takeaway: Retrieval optimizes recall over a huge catalog cheaply; ranking optimizes precision on a small shortlist using the expensive, accurate model — a foundational pattern for large-scale recsys/search.

Real-world use case: Nearly every large-scale search or recommendation system uses this cascade — often with more stages (retrieval → light ranking → heavy ranking → business-logic re-ranking).

Ethics of Recommender Systems

1. **Q:** How could two systems with identical architecture and objective produce opposite social outcomes in different businesses?
2. **Q:** What's a concrete design change to reduce the “engagement maximization → toxic content” loop?

Core takeaway: The algorithm's math is neutral; harm or benefit emerges from what objective it optimizes and the business it's applied to. There is no purely technical fix.

Real-world use case: Documented cases: engagement-optimized social feeds amplifying extreme content; predatory-lending or gambling ads favored by pure click/bid-maximizing ad auctions.

TensorFlow / Keras Implementation Details

1. **Q:** Why normalize v_u and v_m to unit length before the dot product, and how does this change what it measures?
2. **Q:** What's the role of tf.keras.layers.Dot, and why can't a Dense layer combine the two towers instead?

Core takeaway: L2-normalizing before the dot product converts the score into something closer to cosine similarity (direction match) rather than raw magnitude-sensitive dot product — a practical trick that stabilizes training.

Real-world use case: Normalized/cosine-similarity embeddings are standard in production embedding systems (recommenders, semantic search, face recognition) because they're robust to arbitrary scale differences between examples.

7. Tailored Report

Who this is for: you already know Python and have finished Course 1 (Regression/Classification) and are deep into Course 2 (Advanced Learning Algorithms) — this report assumes that baseline, not zero-ML knowledge.

At a glance

This block of 5 lectures is the **second recommender-systems algorithm** in Course 3 (content-based filtering), following collaborative filtering earlier in the same module. It builds directly on your neural-network foundations from Course 2 — the two-tower architecture here is literally two of the dense-layer networks you already know how to build, combined via a dot product instead of a single output unit. It's immediately followed by the module's closing TensorFlow lab and then the course pivots to reinforcement learning.

Foundational vs. supplementary

Priority	What it covers
Foundational	The collaborative → content-based pivot (features + learned v_u, v_m vs. ratings-only)
Foundational	The two-tower architecture: same-dimension outputs, dot product, joint MSE training
Foundational	Why v_m can be pre-computed, and how that one fact enables both “similar items” and the retrieval step — really one idea threaded across two lectures
Supplementary	Specific example feature lists (top-1000-movies indicator, average rating per genre, etc.)
Supplementary	Exact retrieval recipe numbers (10 similar movies, top-10-per-genre, top-20-per-country)

Priority	What it covers
Supplementary	Exact Keras code / variable names — useful as reference, not for memorization
Supplementary	The specific ethics case studies (travel vs. payday loans) — memorable illustrations, not content with a “right answer”

Commonly tricky / misunderstood parts

- The b_j removal and $w \cdot x \rightarrow v_u \cdot v_m$ renaming is easy to skim past, but it's the mechanical bridge from “linear model with parameters” to “two learned embedding vectors.”
- Why there's a *single* joint cost function trips people up — it's tempting to think of the user and movie networks as independent models, but they're two branches of one computational graph trained end-to-end.
- The L2-normalization step is presented almost as a footnote but is a meaningful modeling choice (dot product $\rightarrow$ effectively cosine similarity) — easy to under-appreciate on a single pass.
- The retrieval step's “it's okay to be sloppy” framing can feel counter to ML instinct — the key is that accuracy requirements are *allocated differently* across the two stages by design.

Next steps to solidify this lecture's content

1. **Re-derive by hand** the transition from $w_j \cdot x_i + b_j$ to $v_u^j \cdot v_m^i$, noting exactly what changed (renaming, dropped bias) and what didn't (still linear combination + squared-error loss underneath).
2. **Sketch the full two-tower graph from memory** — both dense stacks, both 32-unit outputs, the L2-norm steps, and the Dot layer — then check it against the Section 5 code.
3. **Re-watch/re-read the “pre-compute v_m overnight” aside and the retrieval step back-to-back** — they're one continuous idea, and seeing them in sequence makes “why retrieval works” click much faster.

Takeaway: This module hands you the second half of the recommender-systems toolkit and, more importantly, a general pattern — twin embedding towers matched by a similarity score — that reappears throughout production ML far beyond movie recommendations.